

Practical Session 1: Crystallography

Objectives

By the end of this session, you will have experience in

1. using VESTA to visualize crystal structures
2. finding crystal structure data for different materials
3. setting up and running a basic CASTEP calculation

About visualization software

There are a number of different software packages which allow the visualization of atomistic structures. In this session we are going to work with the VESTA code <https://jp-minerals.org/vesta/en/> which is freely available for non-commercial users.

At its most basic level, visualization software provides a computational version of the ball-and-stick models lying around 21 Banbury Road, but it has the ability to do much more. For example, we can use the code to

- analyse crystal structures, measuring bond lengths and angles
- generate new model structures, e.g. surfaces, interfaces and defects
- plot data other than atomic positions, e.g. charge densities, isosurface representations of wavefunctions, phonon amplitudes
- and much more

Visualising a simple structure

VESTA is a very powerful tool, but we will start by asking it to do something simple. Opening the application, we are faced with a “blank canvas”. To proceed, we need to generate a New Structure, which is accessed via the File menu. This opens a window with several tabs. We are going to start with where we finished in the

lecture, which is silicon. The first thing we need to do is specify the crystal structure, which we do using the Unit cell tab.

Now we need to tell VESTA that silicon is cubic (left menu) and that its space group is 227. Upon selecting this all the lattice parameter boxes are greyed out except one, which allows us to specify the cubic lattice parameter $a = 5.43$.

If we hit OK at this stage, VESTA will draw us our unit cell. What is missing is the atoms. To get back to the structure menu, we have to go via Edit Data which appears in the Edit menu. We need to go to the Structure Parameters tab, where we can add New atoms. By default, VESTA creates an element XX at the origin. We can change its Symbol to Si, and then hit OK (or Apply if we want to keep making changes in this menu).

Note how there is a choice of origin; see <http://img.chem.ucl.ac.uk/sgp/large/227a.htm>. This affects how we choose our basis.

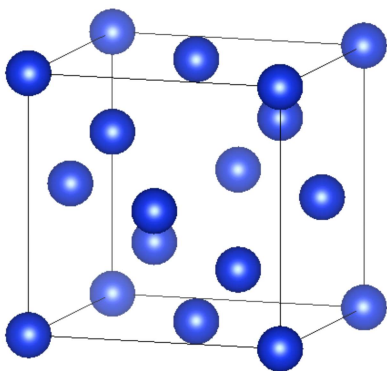


Figure 47: The crystal structure of diamond Si visualised by VESTA

All being well, you should end up with a vaguely familiar crystal structure. You can change the camera position by clicking and dragging, and zooming with the mouse wheel; alternatively, the buttons along the top can be used. Let's note a few things:

- VESTA has drawn the conventional unit cell
- It has included all the atoms on the edges, corners and faces, even though some of them technically belong to different unit cells. However, the atoms that truly belong to this unit cell are shown in a darker colour
- VESTA also automatically generated all the symmetry-equivalent atoms based on the space group we told it

Basic analysis

The buttons on the left allow various quantities to be measured, e.g. the distance between two atoms or the angle between two bonds.

Select the tool, and then click on the atoms that you are interested in. The information will appear in the output box at the bottom of the page. Use this to check the bond length between nearest neighbours, and also that the tetrahedral angle is what you expect.

Changing the appearance

Try investigating the properties box to see if you can change the atoms' colour. We can also get VESTA to draw bonds between atoms, through Edit/Bonds. VESTA does this by searching through all the pairs of atoms and drawing a bond if the distance between them is less than a threshold. You can choose to restrict this search to atoms within the unit cell boundary, or let VESTA form bonds to atoms outside the cell. See if you can reproduce the figure below:

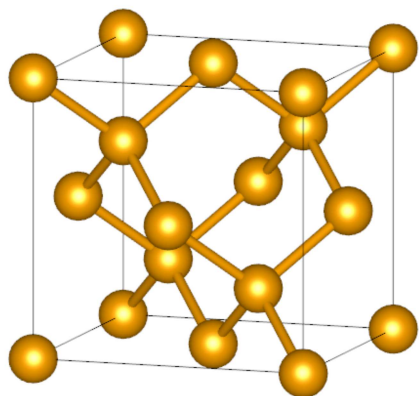


Figure 48: The crystal structure of diamond Si visualised by VESTA, with bonds

Saving a structure and images

If you have spent time creating an artistic masterpiece it is advisable to save it. VESTA has its own file format; if you save your structure as `file.VESTA`, you will be able to return to it at a later time, using VESTA. If you use the File/Open menu you will see that VESTA can read many different file formats, which we will take advantage of below.

For your report, you might choose to include images of the crystal structure. You can do this using the File/Export as raster/vector image, which will produce bitmap-type images (jpeg, tiff etc.) or vector drawings (pdf, eps).

Exercise

The mineral forsterite has the formula Mg_2SiO_4 and crystallises in the orthorhombic space group 62 (Pnma) with $a = 1.02066$ nm, $b = 0.59845$ nm and $c = 0.47536$ nm. You are given the following data

Element	R_i	R_j	R_k	Wyckoff
Mg	0	0	0	4a
Mg	0.2224	0.25	0.5079	4c
Si	0.4066	0.25	0.0734	4c
O	0.4068	0.25	0.7388	4c
O	0.0491	0.25	0.2823	4c
O	0.3373	0.0376	0.22	8d

Use VESTA to visualize this crystal structure, and answer the following questions:

1. How many atoms are in the conventional unit cell?
2. What is the nearest neighbour distance between a) Si and O and b) Mg and O?
3. Now tell VESTA to draw the Si-O and Mg-O bonds (unlike for Si you have to specify the chemical species of the two atoms making the bond). Switch to the Polyhedral style. What sort of co-ordination polyhedra (e.g. MgO_n) are formed? Describe how the different polyhedra are connected to each other in terms of face/edge/corner sharing.
4. Produce an image which (in your opinion) provides the clearest picture of this crystal structure

Reading in structures from files

You might have found typing in atomic co-ordinates quite tedious. Luckily VESTA is able to read in structures from a variety of data formats, including the CIF files discussed in the lecture. Let's try one of the materials covered in the lecture, pentacene. You can download a CIF file from the [Crystallography Open Database](#) by searching for entry 4109834. Then it is a simple case in VESTA of using File/Open to read in this file. Hopefully you should find VESTA will draw a nice picture of the pentacene crystal. Furthermore if you examine the Unit cell or Structure parameters you will find all the data about the crystal structure which VESTA has extracted from the file. You can also open the CIF file using a text editor to see where this information has come from

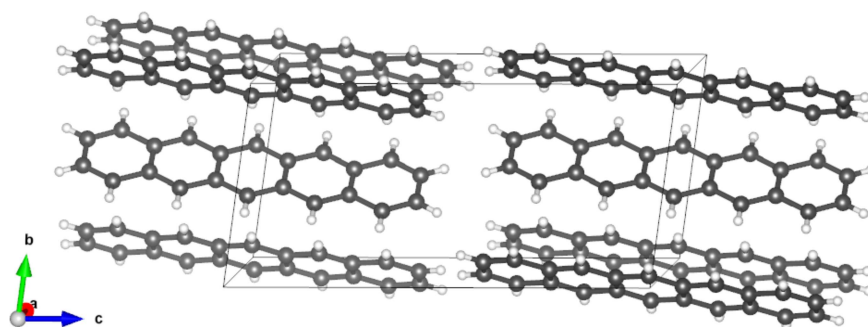


Figure 49: A pentacene crystal visualized by VESTA, using a CIF file containing experimental data *JACS* **129**, 10316 (2007)

Fig. 49 shows a VESTA visualization. Note in this case it is a good idea to plot atoms inside and outside the unit cell so that we always see the five rings of pentacene. VESTA is smart enough to do this for us.

Note that, if you have imported a file, if you want to save your work it is a good idea to Save as a new vesta file, compared to overwriting your original version.

You might be wondering what we would do if we wanted to model an isolated (gas phase) pentacene molecule, compared to a crystal. If you look again under the Unit cell menu you should find there is a Molecule system, which disables the periodicity. Then, the Structural data simply corresponds to the absolute positions (in Å) of each atom. As a very simple example you can try setting up a CO molecule in VESTA (bond length 1.13 Å).

As for crystals, typing in co-ordinates by hand can be slow and it is preferable to be able to import data. Probably the simplest file format one can use to describe a molecule is the xyz format. As it sounds, the xyz file simply gives the x, y, z Cartesian co-ordinates of each atom which makes up the molecule. You can find a pentacene.xyz file on Canvas. If you examine it in a text editor, you will find the first line gives the number of atoms in the molecule, the second is a comment which does not affect the plotting, and then the co-ordinates are given in Å. Again, VESTA can read the xyz file. Try reading in the pentacene file and measuring some bond lengths and angles.

Exercise

Below is a list of materials and a brief description. Your job is to use VESTA to produce visualizations of their crystal structure, and answer the questions.

You are encouraged to look for CIF files and import these, making sure you cite the original source of the data (e.g. the paper where it was first reported). If searching the COD you might try using the option to search by the “Hill formula”. For instance to search for pentacene you would type

C22 H14

(note the space). The rule for ordering the elements in the Hill formula is to start with carbon, then hydrogen, then alphabetical.

Sometimes it will be useful to visualize more than one unit cell. The way to control this is to through `Objects/Boundary`. By default, the boundary will plot fractional co-ordinates in the range $0 < x < 1$, but by increasing this you will plot atoms outside the first cell. You can also specify cutoff planes through their Miller indices so that the visualization region doesn't necessarily need to correspond to the shape of the unit cell.

1. Magnesium diboride (MgB_2) - a hexagonal layered superconductor. What is the distance between the Mg and B layers?
2. Yttrium aluminium garnet (YAG) - a cubic material with formula ($\text{Y}_3\text{Al}_5\text{O}_{12}$) which, when doped, is used in lasers. Describe the co-ordination polyhedra
3. The **tetragonal** phase of cesium tin iodide (CsSnI_3) which is an example of a perovskite-type light absorber for use in photovoltaics. How do the SnI_6 octahedra fit together? (it is easier to visualize the structure if you remove bonds involving Cs atoms)
4. Neodymium iron boron ($\text{Nd}_2\text{Fe}_{14}\text{B}$) - a permanent magnet material. How many Nd atoms are in the unit cell?

Setting up and running a CASTEP calculation

We are now going to actually run an atomistic modelling calculation. Specifically, we are going to set up a density-functional theory calculation which will calculate the ground state electron density and total energy. We are going to use the CASTEP (<http://www.castep.org/>) software package to this. You have already met CASTEP in your introduction to modelling course. However, in that case you interacted with CASTEP via the python-based Atomic Simulation Environment ([ASE](#)). Here we are going to work with CASTEP directly using the command line interface. Although this involves a steeper initial learning curve, it unlocks access to a number of features required to perform more sophisticated calculations.

The first step is to log onto one of the central computers using the procedure described in the earlier session. This places you in your home directory:

```
~$
```

It is a good idea to keep all your files organised using a sensible directory structure. Here we show making a directory called `01_crystal` to keep all of the files concerning today's practical together, but you might prefer to use a different directory:

```
~$ mkdir p01_crystal
```

We then move into this directory:

```
~$ cd p01_crystal
```

It is mandatory that atomistic modellers know how to calculate the properties of diamond silicon, and this will be our first example. Again, we store all these calculations in one directory.

```
~/p01_crystal$ mkdir 01_Si
```

```
~/p01_crystal$ cd 01_Si/
```

As discussed in the lecture, in order to perform the calculation we need to tell CASTEP what the crystal structure is, and this is done through a `cell` file. Later we will see how to generate these using a special tool, but first we will make one ourselves. Use a text editor like `nano` to create a file called `si.cell` which looks like the following:

Hint: if you start typing `cd p01` and then hit the Tab key, the command line interpreter will try to autocomplete the entry

e.g. type `nano si.cell`

```
! Specify the lattice
! For FCC, unit cell length is a/sqrt{2}
%BLOCK LATTICE_ABC
3.84 3.84 3.84
60 60 60
%ENDBLOCK LATTICE_ABC

! Now the (fractional) co-ordinates of the atoms
%BLOCK POSITIONS_FRAC
Si 0.0 0.0 0.0
Si 0.25 0.25 0.25
%ENDBLOCK POSITIONS_FRAC

! Analyse the symmetry
SYMMETRY_GENERATE

! Specify the k-point sampling
KPOINT_MP_GRID 4 4 4
```

In the lecture, we discussed most of the ingredients of this file. Note that every line which begins with an exclamation mark `!` is a comment. As usual in computer programming, these comments are ignored by CASTEP but help the humans understand what is going on. The `%BLOCK` syntax tells CASTEP that we are specifying some specific information, and CASTEP will keep reading this information until it hits an `%ENDBLOCK` line.

We can also use the hash symbol to specify a line is a comment

The two blocks specify the lattice and atomic positions. The line `SYMMETRY_GENERATE` is important to include because it greatly improves the efficiency of the calculation, for reasons described shortly. The last line specifies the size of the grid of k -points used in the calculation; this is using a $4 \times 4 \times 4$ grid. Naively we might expect this would involve solving the Schrödinger-type equation at 64 different k -points, but in reality the number is much smaller. This is where the symmetry comes in: it turns out that thanks to the real space symmetry of the crystal, there is also symmetry in reciprocal space. Specifically if two different k -points are related to each other by a symmetry operation of the crystal, we only need to solve the Schrödinger equation for one of them. In the end, we therefore only work with a smaller number of “irreducible” k -points which cannot be mapped onto each other through symmetry operations of the crystal. The `SYMMETRY_GENERATE` line ensures this reduction is performed.

This file is actually all we need to do the calculation! It is important we are in the same directory as the file. We can check this using the `ls` command and making sure the file is listed:

```
~/p01_crystal/01_Si$ ls
si.cell
```

It is exceptionally bad practice to run CASTEP without providing a `param` file as well as a `cell` file, and you should never do it again! We’re doing it here to try to keep things as simple as possible.

We now run CASTEP by typing the command

```
~/p01_crystal/01_Si$ castep.serial si
```

The computer will then think for a very short while, and then return you to the command line. Typing `ls` again will show a number of files have been generated:

```
~/p01_crystal/01_Si$ ls
si.bands  Si_C19_LDA_0TF.usp  si.castep_bin  si.check
si.bib    si.castep             si.cell        si.cst_esp
```

Let’s consider these:

- `si.castep_bin`, `si.check` and `si.cst_esp` are all binary files which CASTEP can understand, but we cannot. However, we can if we want use CASTEP and its associated tools to extract information (we will not at this stage).

- `si.bib` will be useful when you are writing your report: it is a BibTeX file containing key papers which describe the theory implemented in CASTEP.
- `Si_C19_LDA_0TF.usp` is interesting: this is the *pseudopotential* for Si, which CASTEP has generated automatically. This file can be read by humans, and the first few lines contains a useful summary: after that however, there are lots of numbers which gives CASTEP all the information about the pseudopotential.
- `si.bands` contains key information about the solutions of the Schrödinger-like “Kohn-Sham” equations at the different k -points, which we would identify as the bandstructure. We will have more to say about this in a later practical session
- `si.castep` is the output file, which provides a log of the calculation. This is the most useful file for today!

Examine this file, e.g. using the `less` command:

```
less si.castep
```

The output file can be split into three parts. The first bit is initial information that tells us how the calculation is set up, consisting of:

1. Generic information about the software and how it was installed
2. Information about the creation of the pseudopotentials
3. Some details about the parameters that can be set to control the calculation
4. Structural information
5. The k -points used and the symmetry found
6. Estimates of the computational resources that will be required to run the calculation

The next bit, starting from “Calculating total energy”, is describing the actual density-functional theory calculation. In DFT, the density of the electrons in their ground state is found by solving the Kohn-Sham equations *self-consistently*: that means the electron wavefunctions which solve the Kohn-Sham equations correspond to the same density which was used to generate the Kohn-Sham potential appearing in those equations. In practice, this density is found iteratively: we guess a density, build and solve the Kohn-Sham equations and see whether the density that comes out is the same as what we put in. If not, we mix the old and new densities together somehow and solve the equations again. Each time we do this is referred to as

a step or an iteration in the SCF (self-consistent field) cycle. At each iteration, CASTEP outputs some information, including how many iterations we have performed so far and the total energy. When the input and output density are the same to within some tolerance the SCF cycle has converged and the calculation has finished. This leads us onto the final part of the output file, which begins with Final energy, E and gives us information about what has been calculated. This is of course why we did the calculation in the first place, and as the week progresses we will talk more about these outputs. For now, we can just focus on the one number E which gives us the total energy (electrons and nuclei) when the electrons are in their ground state.

Note that saying an SCF cycle has “converged” means something different than the convergence discussed in the lecture, which referred to making sure a sufficient number of k -points and plane waves were used in the calculation.

From reading `si.castep`, answer the following questions:

1. Which exchange-correlation functional did the calculation use?
2. What is the number of electrons in the unit cell (is this what you might expect?)
3. What is the unit cell volume? Verify it equals $a^3/4$, where a is the cubic lattice parameter
4. How many k -points were used in the calculation?
5. How many SCF iterations were required to reach convergence?
6. What is the final calculated total energy?
7. What was the total time spent on this calculation? How much of this was spent initialising, compared to actually running, the calculation?

Note that in this calculation, we didn’t specify any parameters; CASTEP used the default values for everything. We will see how to set parameters in the next session.

Generating cell files from CIF files

If we don’t want to generate a cell file by hand, but we have a CIF file, there is a useful Python-based tool we can use called `cif2cell`. Navigate back to your `p01_crystal` directory and make a new directory called `02_hexBN`.

```
~/p01_crystal$ mkdir 02_hexBN
~/p01_crystal$ cd 02_hexBN/
```

On Canvas you can find the file `hexBN.cif`. You can download this file onto your own computer and look at it in VESTA. But now you

need to copy this CIF file from your computer onto the remote machine. How you do this depends on your operating system, and at this point you should consult the guides elsewhere on copying files.

Once you have moved the cif file to the directory, we can now use `cif2cell`. This code can be used to generate structure files for a number of electronic structure codes, not just CASTEP; to find out more about it you can run

```
$ cif2cell --help
```

For our purposes, to generate a CASTEP cell file we type

```
~/p01_crystal/02_hexBN$ cif2cell -p castep hex_bn.cif
```

All being well, this should generate a file `BN.cell`. If you examine this file, you will see it has familiar lines like the `LATTICE` and `POSITIONS` blocks, but also quite a lot of other text; lots of comments that CASTEP will ignore, but also things like symmetry operations that we don't need to specify since CASTEP will find these itself. So, it is necessary to edit this file a bit: you can delete all the lines apart from the two blocks, and then add in lines telling CASTEP to check the symmetry and specifying the k-point grid:

```
%BLOCK LATTICE_CART
ang      # angstrom units
  2.168518950822196  -1.2519950000000000  0.0000000000000000
  0.0000000000000000  2.5039900000000000  0.0000000000000000
  0.0000000000000000  0.0000000000000000  6.6612000000000000
%ENDBLOCK LATTICE_CART

%BLOCK POSITIONS_FRAC
N   0.3333333333333333  0.6666666666666667  0.7500000000000000
N   0.6666666666666667  0.3333333333333333  0.2500000000000000
B   0.3333333333333333  0.6666666666666667  0.2500000000000000
B   0.6666666666666667  0.3333333333333333  0.7500000000000000
%ENDBLOCK POSITIONS_FRAC

SYMMETRY_GENERATE = TRUE

KPOINT_MP_GRID 4 4 2
```

You can then run this calculation with the command

```
$ castep.serial BN
```

which still should be quite fast.

Exercise

Your task now is to set up and run a total energy calculation for cesium chloride, CsCl. Use a lattice constant of 4.123 Å and a k -point sampling of $4 \times 4 \times 4$. You might choose to look for a CIF file, but it might be quicker to write the cell file yourself. Good luck!